1/3rd term Programming Fundamentals

Oct. 9 2026, 18:30-21:30h

- You can solve the problems in any order. Solutions must be submitted to the automated judgement system Themis. For each problem, Themis will test ten different inputs, and check whether the outputs are correct.

- Grading: You get one grade point for free. The remaining nine points are based solely on the judgement given by Themis. The first problem is worth one grade point. The remaining four problems are worth two grade points each.

- You can score partial points if a program passes several but not all test cases. For example, if you pass 6 out of 10 test cases, then you are awarded 60% of the grade points for that exercise.

- Inefficient programs may be rejected by Themis. In such cases, the error will be 'time limit exceeded'. The time limit for each problem is one second.

- The number of submissions to Themis is not limited. No points are subtracted for multiple submissions.

- There will be no assessment of programming style. However, accepted solutions are checked manually for cheating: For example, precomputed answers will not be accepted, even though Themis accepts them.

- Needless to say, you are not allowed to work together. If plagiarism is detected, both parties (supplier of the code and the person that sends in copied code) will be excluded from any further participation in the course.

- You are not allowed to use email, phones, smartwatches, tablets, calculators, etc. There is a calculator available on the exam computers (see icon on the desktop). You are allowed to consult the ANSI C book and a dictionary. You are not allowed to use a printed copy of the reader or the lecture slides, however they are available digitally (as a pdf) in Themis. You are allowed to access your own submissions previously made to Themis.

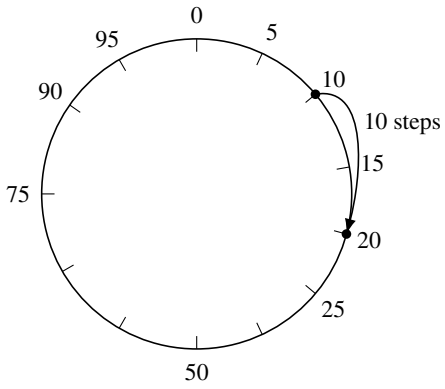
- For each problem, the first three test cases (input files) are available on Themis. These input files, and the corresponding output files, are called `1.in`, `2.in`, `3.in`, `1.out`, `2.out` and `3.out`. These files can be used to test whether the output of your program matches the requested layout, so that there can be no misunderstanding about the layout and spaces in the output.

- **If you fail to pass a problem for a specific test case, then you are advised not to lose much time on debugging your program, and continue with another problem. In the last hour of the exam, all test cases will be made visible in Themis.**

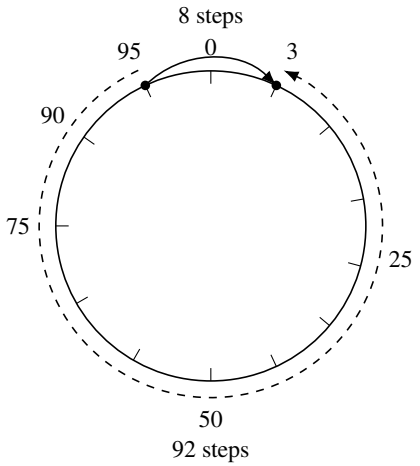
Problem 1: Circular Distance

Consider the integers from 0 to 99 arranged clockwise on a circle, in increasing order. The number 0 follows immediately after 99. From any number, one step can be made in either direction (clockwise or counterclockwise) to one of its two neighbouring numbers.

For example, the distance between 10 and 20 is 10, as illustrated below (left image). The distance between 95 and 3 is 8 (right image), since moving in clockwise direction we pass the 8 numbers 96, 97, 98, 99, 0, 1, 2, 3. Moving in the opposite direction would require 92 steps, which is clearly longer.



Distance from 10 to 20



Distance from 95 to 3

The input consists of two integers a and b, where $0 \leq a < 100$ and $0 \leq b < 100$. The output must be a single integer: the minimum number of steps required to move from a to b.

Example 1:

input:
```
10 20
```
output:
```
10
```

Example 2:

input:
```
95 3
```
output:
```
8
```

Example 3:

input:
```
3 95
```
output:
```
8
```

Problem 2: Bounce to Zero

Consider the integer interval $[0..N]$, containing all integers from 0 up to and including N. A ball starts at position 1 and initially moves to the right. In each step, the ball moves exactly d positions.

When the ball reaches an end of the interval, it bounces and continues in the opposite direction. If a step would move the ball beyond an end of the interval, the remaining part of the step is made in the opposite direction. For example, if $N = 10$ and $d = 3$, the positions are

$$1 \to 4 \to 7 \to 10 \to 7 \to 4 \to 1 \to 2 \to 5 \to 8 \to 9 \to 6 \to 3 \to 0.$$

Thus, it takes 13 steps to reach position 0.

The input for this problem consists of the two integers N and d, where $1 \leq d < N \leq 10000$. Write a program that determines the number of steps required for the ball, starting at position 1 and initially moving to the right, to reach position 0 for the first time. Note that the test cases have been chosen such that position 0 will eventually be reached.

The output must be a single integer: the number of steps required to reach position 0 for the first time.

Example 1:

input:
```
10 3
```
output:
```
13
```

Example 2:

input:
```
5 1
```
output:
```
9
```

Example 3:

input:
```
5 3
```
output:
```
3
```

Problem 3: Reverse multiple

The reverse of a positive integer is obtained by writing its digits in reverse order. For example, the reverse of 123 is 321, and the reverse of 120 is 21. Leading zeroes are ignored.

Given integers $n > 0$, and a and b (where $1 \leq a < b$), find the smallest integer k in the interval $[a..b]$ for which $\text{reverse}(k \cdot n) = k \cdot \text{reverse}(n)$.

For example, consider $n = 15$ and $2 \leq k \leq 20$. For $k = 11$ we find

$$\text{reverse}(11 \cdot 15) = \text{reverse}(165) = 561 = 11 \cdot 51 = 11 \cdot \text{reverse}(15).$$

No smaller value of k in the given interval has this property, so the answer is 11.

Write a program that finds the smallest value of k satisfying the condition. If there is no such value, output `NONE`. The input consists of three integers n, a, and b, where $1 \leq n \leq 10000$ and $2 \leq a \leq b \leq 1000$. The input is chosen such that all intermediate computations fit in a 32-bit signed integer. The output must be the smallest integer k in $[a..b]$ satisfying the condition, or `NONE` if no such integer exists.

Example 1:

input:
```
123 2 10
```
output:
```
2
```

Example 2:

input:
```
15 2 20
```
output:
```
11
```

Example 3:

input:
```
15 12 20
```
output:
```
NONE
```

Problem 4: Removing extrema

Consider an array of integers. An element is called an extremum if it has two neighbours and is either strictly greater than both of them or strictly smaller than both of them. The first and last elements of the array have only one neighbour and are therefore never extrema.

We now consider a series of reductions. In one reduction step, all extrema are removed from the array at the same time. The remaining elements keep their original order.

For example, in the the array `[1,2,4,5,3]`, the element 5 is an extremum, so the first reduction step gives `[1,2,4,3]`. In this new array, 4 has become an extremum. A second reduction step therefore gives `[1,2,3]`. This array contains no extrema and is therefore stable.

Write a program that reads an array from the input, and repeatedly performs reduction steps until the array becomes stable. The first line of the input contains an integer n, where $1 \leq n \leq 1000$. The second line contains the array containing n integers, each between 0 and 10000. The output must be the stable array. The elements must be enclosed in square brackets and separated by commas, without spaces.

Example 1:

input:
```
5
1 2 4 5 3
```
output:
```
[1,2,3]
```

Example 2:

input:
```
5
1 2 3 4 5
```
output:
```
[1,2,3,4,5]
```

Example 3:

input:
```
7
1 4 3 5 6 2 7
```
output:
```
[1,5,7]
```

Problem 5: Longest circular common segment

A *segment* of an array consists of consecutive elements of the array. In this problem, arrays wrap around. For example, in the array `[1,2,3,4,5]` the sequence `[4,5,1,2]` is a segment because it wraps around from 5 to 1.

You are given two arrays. A *common segment* is a segment that occurs in both arrays. The segment may start at any position in either array and may wrap around in either or both arrays.

A common segment may contain at most as many elements as the shorter of the two arrays. For example, consider the arrays `[1,2,3,4,5]` and `[4,5,1,2,8]`. The sequence `[4,5,1,2]` is a common segment. In the first array it wraps around, while in the second array it does not.

Write a program that finds a longest common segment of the two arrays. If there are several longest common segments, output the one whose starting position in the first array is smallest. If the two arrays have no common segment, output `[]`.

The first line of the input contains two integers n and m, where $1 \leq n, m \leq 500$. The second line contains n integers, the elements of the first array. The third line contains m integers, the elements of the second array. Each array element is between 0 and 10000.

The output must contain the longest common segment, enclosed in square brackets and with consecutive elements separated by commas. Do not print spaces between the elements.

Example 1:

input:

```
5 5
1 2 3 4 5
4 5 1 2 8
```

output:

```
[4,5,1,2]
```

Example 2:

input:

```
5 5
1 2 3 4 5
8 9 1 2 7
```

output:

```
[1,2]
```

Example 3:

input:

```
3 2
1 2 3
4 5
```

output:

```
[]
```